

FACULTAD DE ECONOMÍA, EMPRESA Y NEGOCIOS.
Metodologías de Desarrollo de Software 0

PLAN DE SOPORTE, MANTENIMIENTO Y CONSOLIDACIÓN

de Arquitectura y Base de Datos de SumaControl

Catedrático/a:
Lcda. Ana Lissette Girón de Bermúdez

Presentado por:
Victor Eduardo Alonzo López
Samuel Rafael Martínez Ramírez
Gonzalo Ricardo Guzmán Sosa

Actividad Asíncrona Grupal — Semana 15
Antiguo Cuscatlán, Noviembre de 2026

CONTROL DEL DOCUMENTO

Documento	Plan de Soporte, Mantenimiento y Consolidación de Arquitectura/BD de SumaControl
Actividad	Actividad Asíncrona Grupal — Semana 15 (MDS 0)
Versión	1.0 — Noviembre de 2026
Equipo	Grupo N.º 1, Sección 1-1 — Alonzo López (PO), Martínez Ramírez (SM), Guzmán Sosa (Dev)
Base técnica	Repositorio oficial, rama develop más las ramas feature/sprint6-cola-prioridad (Sprint 6) y sprint7-victor (Sprint 7, roles dinámicos)
Repositorio	https://github.com/202401579-hue/Proyecto-Integrador
Carpeta de entregables	https://github.com/202401579-hue/Proyecto-Integrador/tree/develop/docs/arquitectura-y-datos

1. INTRODUCCIÓN Y ALCANCE

Este informe reúne tres cosas que, en un sistema en operación, dependen unas de otras: cómo se atiende un problema cuando ocurre (soporte), cómo se mantiene el sistema para que ocurran menos (mantenimiento) y qué es exactamente lo que se está manteniendo (arquitectura y datos). Quien recibe un incidente de madrugada necesita saber a quién escalarlo, pero también en qué nodo corre cada pieza y en qué colección vive cada dato.

Por eso las Secciones 2 y 3 definen los procesos de soporte y de mantenimiento, y las Secciones 4, 5 y 6 consolidan la arquitectura, su relación con los artefactos previos del proyecto y el modelo de datos. Todo lo descrito corresponde a lo que está implementado en el repositorio; donde el sistema difiere de lo que la consigna menciona de forma genérica (PostgreSQL, WebSockets, schema.sql), se indica la adaptación, aprobada por la docente en su aclaración de entregables, y la Sección 7 consigna lo que aún no existe.

2. SOPORTE TÉCNICO Y NIVELES DE ATENCIÓN

El soporte se organiza en tres niveles. Cada nivel resuelve lo que puede resolver con su alcance y escala lo demás con el ticket completo; así el usuario habla una sola vez y cada problema llega a quien tiene las herramientas para resolverlo.

2.1 Niveles de atención y tiempos de respuesta

Nivel y responsable	Alcance	Ejemplos en SumaControl	Canal	SLA resp.	SLA solución
L1 — Mesa de ayuda y soporte en garita MDS 0 / Operador de garita líder	Atención inicial al usuario final: dudas operativas de la interfaz, reseteo de claves y guía en el registro de una llegada.	• Olvido de contraseña. • Duda al registrar la llegada de un camión. • Orientación sobre bahía libre u ocupada.	Ventanilla de garita, chat interno, ticket L1	< 15 min	< 30 min
L2 — Infraestructura y redes GSO 0 (servidores) / GRE 1 (redes)	Conectividad de la VLAN de garita, disponibilidad del servidor Node.js (frontend y API) y del servidor MongoDB.	• Pérdida de señal en el puesto de garita. • La API responde 500 o no responde. • mongod inalcanzable desde el servidor API.	Escalamiento desde L1, alerta de monitoreo	< 30 min	< 2 h
L3 — Mantenimiento de código y algoritmos PON 0 (API e interfaz) / EDA 0 (motor de cola)	Corrección de defectos en backend y frontend, reglas de negocio (RN-04 tolerancias, RN-08 exclusividad del gateway 5) y el motor de cola con prioridad y aging.	• RN-04 no clasifica bien un TARDÍO. • Asignación inconsistente en el gateway 5. • Orden incorrecto en la cola con prioridad.	Escalamiento desde L2, GitHub Issues	< 1 h	< 4 h (hotfix)

2.2 Matriz RACI

La matriz reparte la responsabilidad de cada actividad del soporte. Cada fila tiene un único aprobador (A), para que nunca haya duda sobre quién responde por el resultado.

Actividad	L1 MDS 0	L2 GSO 0 / GRE 1	L3 PON 0	L3 EDA 0	PO / SM
Recibir, registrar y clasificar el ticket por severidad	A/R	C	I	I	I
Orientar al usuario o resetear su contraseña	A/R	—	—	—	—
Diagnosticar red/VLAN, servidor Express o base de datos	I	A/R	C	—	I
Activar la contingencia (registro manual, asignación manual de bahía)	R	A	C	I	I
Ejecutar el rollback de versión (RTO < 15 min)	I	R	A	—	I
Corregir un defecto en backend o frontend (bugfix / hotfix)	I	I	A/R	C	I
Corregir u optimizar el motor de cola (prioridad y aging)	—	—	R	A	I
Aprobar y desplegar el hotfix (revisión de código y regresión)	I	R	R	C	A

Actividad	L1 MDS 0	L2 GSO 0 / GRE 1	L3 PON 0	L3 EDA 0	PO / SM
Mantenimiento preventivo (respaldos, logs, índices)	I	A/R	C	—	I
Mantenimiento adaptativo (SO, navegador, IP, dependencias)	I	A/R	R	—	I
Comunicar el estado al usuario y cerrar el ticket	A/R	I	I	I	I
Registrar la mejora en el Product Backlog (evolutivo)	R	I	C	C	A

R = Responsable de ejecutar · A = Aprobador final (uno por actividad) · C = Consultado · I = Informado.

2.3 Matriz de severidad y acuerdos de nivel de servicio

La severidad se asigna por el impacto en la operación de la garita, no por la dificultad técnica del problema: un fallo simple que detiene el registro de camiones es S1; uno complejo en un módulo que nadie usa esta semana es S3.

Sev.	Criterio e impacto	Nivel	Ejemplo en SumaControl	SLA atención	SLA solución	Contingencia	Responsable
S1 Crítico	Sistema caído o inaccesible: se detiene el registro de camiones en garita.	L2 / L3	Caída de mongod o del servidor Express en hora pico.	< 15 min	< 1 h	Registro manual en bitácora de papel y rollback de versión (RTO < 15 min).	GSO 0 / PON 0
S2 Alto	Falla grave en una función principal sin detener el ingreso general.	L3	RN-04 clasifica mal la puntualidad o falla la asignación del gateway 5.	< 30 min	< 4 h	El Coordinador asigna la bahía a mano desde el tablero de gateways.	PON 0 / EDA 0
S3 Medio	Inconsistencia menor o lentitud en módulos que no bloquean la operación.	L2 / L3	Lentitud en el historial de eventos o en los indicadores.	< 2 h	< 24 h	Revisar la consulta con explain(), reconstruir índices y reiniciar el proceso Node.	GSO 0 / PON 0
S4 Bajo	Mejora cosmética o duda del usuario sin impacto operativo.	L1	Cambiar la etiqueta de un botón de la interfaz.	< 4 h	< 72 h	Se registra en el Product Backlog evolutivo para el siguiente Sprint.	MDS 0

2.4 Canales de reporte y flujo de escalamiento

1. Reporte. El usuario avisa en la ventanilla de garita, por el chat interno o abre un ticket L1. Debe indicar pantalla, hora aproximada y número de pedido o de gateway involucrado.

2. Clasificación. L1 asigna la severidad S1–S4 con la matriz anterior. Ante la duda entre dos niveles se toma el más grave, porque subir un ticket es barato y bajarlo tarde no.

3. Resolución en L1. Si es una duda de uso o una clave olvidada, L1 la resuelve y cierra dentro de su SLA. Si no, escala con el ticket completo para no obligar al usuario a repetir su historia.

4. Escalamiento. L1 → L2 si el síntoma apunta a red, servidor o base de datos (la API no responde, el tablero no carga). L2 → L3 si la infraestructura está sana pero el comportamiento es incorrecto: es un defecto de código o de regla.

5. Cierre y registro. Quien resolvió documenta causa y solución. Si el incidente expuso un defecto, se abre un issue en GitHub y se enlaza al ticket; si fue S1 o S2, se hace una revisión posterior en la siguiente reunión del equipo.

2.5 Plan de contingencia y rollback (RTO < 15 minutos)

Ante un incidente S1 el objetivo no es encontrar la causa sino restablecer el servicio. El procedimiento está ordenado para que la ruta más probable (una falla tras un despliegue) se resuelva dentro del tiempo objetivo de recuperación.

Paso	Acción	Detalle
1	Declarar el incidente S1	L1 avisa a L2 y a L3 y activa el registro manual en garita.
2	Confirmar el nodo afectado	L2 consulta GET /api/health en el servidor API y verifica la conexión con mongod.
3	Volver a la versión anterior	Si la falla apareció tras un despliegue: checkout del tag anterior, npm ci, npm run build y reinicio del proceso. Es la ruta que debe caber en 15 minutos.
4	Restaurar datos solo si hace falta	Si la causa es la base de datos, se restaura el último volcado (mongodump) en un servidor mongod sano. Si un validador $jsonSchema bloquea escrituras legítimas, se relaja con collMod (validationLevel: "off") mientras se corrige.
5	Verificar	Respuesta 200 de /api/health, inicio de sesión de prueba y registro de una llegada de prueba sobre un pedido de ensayo.
6	Cerrar y comunicar	L1 avisa a garita, se vuelcan al sistema los registros hechos a mano y se abre el issue de causa raíz.

3. PLAN DE MANTENIMIENTO DE SOFTWARE

El mantenimiento se divide en cuatro dimensiones según su motivo. Cada una tiene su propio disparador, su procedimiento y su criterio de éxito, y se apoya en los comandos y la configuración que el repositorio ya tiene.

3.1 Mantenimiento correctivo

Objetivo	Reparar defectos que llegaron a producción.
Disparador	Ante un ticket S1 o S2 que L2 descarta como problema de infraestructura.
Procedimiento	1) Reproducir el fallo con los datos del ticket. 2) Crear la rama bugfix/<ticket> desde develop (hotfix/<ticket> desde main si es S1). 3) Escribir primero una prueba Jest que falle por ese defecto. 4) Corregir. 5) Ejecutar npm test y npm run test:funcional. 6) Pull request con revisión de código de otro integrante. 7) Despliegue y verificación con GET /api/health y el caso original.
Criterio de éxito	Regresión completa en verde (npm test) y el caso reportado reproducido sin el fallo.
Responsable	PON 0 / EDA 0

3.2 Mantenimiento adaptativo

Objetivo	Mantener el sistema funcionando cuando cambia su entorno.
Disparador	Mensual, y ante cambio de navegador, sistema operativo, versión de Node.js o dirección IP.
Procedimiento	La configuración que depende del entorno vive en variables, no en código: PORT, MONGO_URI y JWT_SECRET en el backend; FRONTEND_URL (origen permitido por CORS) y NEXT_PUBLIC_API_URL en el frontend. Un cambio de IP se resuelve editando esas variables y reiniciando. Cada mes se ejecuta npm audit, se revisan las dependencias mayores (Next.js, Express, Mongoose) y se prueba el flujo de garita en la versión vigente del navegador.
Criterio de éxito	Cero vulnerabilidades altas o críticas y el flujo de llegada completo en el navegador de garita.
Responsable	GSO 0 / GRE 1

3.3 Mantenimiento evolutivo / perfectivo

Objetivo	Mejorar rendimiento y usabilidad sin cambiar las reglas de negocio.
Disparador	Al cierre de cada Sprint, sobre el backlog priorizado por el PO.
Procedimiento	Meta de latencia de la API REST: menos de 200 ms. Se mide cada endpoint de lectura del tablero y de la cola; los lentos se revisan con explain() de MongoDB, se ajustan los índices compuestos (ya existen para pedidos y eventos) y se usan consultas lean() cuando no hace falta el documento Mongoose. En el frontend se ajusta el intervalo de sondeo (5 s en la cola, 15 s en el tablero) y se atienden los comentarios de usabilidad del Plan de Capacitación.
Criterio de éxito	Latencia por endpoint por debajo del umbral, comparada antes y después del cambio.
Responsable	PON 0 / EDA 0

3.4 Mantenimiento preventivo

Objetivo	Detectar y evitar fallas antes de que afecten a la operación.
Disparador	Diario (respaldo), semanal (auditoría de la bitácora e índices) y mensual (prueba de restauración).

Procedimiento	Respaldo diario con mongodump comprimido y retención de 14 días. Prueba mensual de restauración en un servidor aparte: un respaldo que nunca se restauró no es un respaldo. Auditoría semanal de la colección eventos (solo inserción) buscando acciones inesperadas. Revisión de índices con db.collection.stats() y reconstrucción si hay degradación. Rotación de logs del proceso Node. Vigilancia de memoria del proceso: como el motor de cola reconstruye su heap en cada consulta, un crecimiento sostenido de memoria indicaría una fuga, no un uso normal.
Criterio de éxito	Respaldo íntegro y restaurable, y sin crecimiento sostenido de memoria entre reinicios.
Responsable	GSO 0 / MDS 0

3.5 Estado actual de las capacidades de soporte y mantenimiento

Esta tabla separa lo que el proyecto ya tiene de lo que este plan define como trabajo por hacer, para que el plan no se lea como una descripción de capacidades inexistentes.

Capacidad	Estado	Detalle
Pruebas automatizadas (Jest) y funcionales	Existe	npm test y npm run test:funcional en backend/.
Verificación de salud	Existe	GET /api/health responde {estado: "ok"}.
Variables de entorno por nodo	Existe	backend/.env.example y NEXT_PUBLIC_API_URL.
Bitácora de auditoría append-only	Existe	Colección eventos con 13 acciones.
Convención de ramas bugfix/ y hotfix/	Por adoptar	El repositorio usa feature/ y sprintN-nombre; se agrega bugfix/ para el soporte.
Respaldos automáticos (mongodump)	Por implementar	No hay scripts en el repositorio; definido en este plan.
Supervisor de procesos y monitoreo	Por implementar	PM2 o systemd; hoy el servidor se levanta con npm start.
Medición de latencia por endpoint	Por implementar	Se propone un middleware de tiempos; no existe todavía.

4. ARQUITECTURA E INFRAESTRUCTURA

4.1 Estilo arquitectónico

SumaControl sigue una arquitectura cliente-servidor organizada en capas. El navegador del usuario ejecuta una aplicación React servida por Next.js; esa aplicación consume una API REST en Express, que a su vez persiste en MongoDB a través de Mongoose. La comunicación entre cliente y servidor es siempre iniciada por el cliente (petición y respuesta): el servidor no empuja datos, y por eso el tablero y la cola se mantienen al día consultando cada cierto tiempo (polling).

Se evaluaron y descartaron otros estilos porque el código no los respalda: no hay un bus de eventos ni consumidores asíncronos (la colección eventos es una bitácora de auditoría escrita en la misma petición, no un canal de mensajería), no hay servicios desplegables por separado, y el backend es un único proceso. La modularidad interna por capas es suficiente para el tamaño del sistema y mantiene el costo de operarlo bajo.

4.2 Vista de arquitectura en 4 capas

Cada capa depende solo de la inmediatamente inferior. Esa regla es lo que permite probar las reglas de negocio sin servidor y cambiar la interfaz sin tocar la lógica.

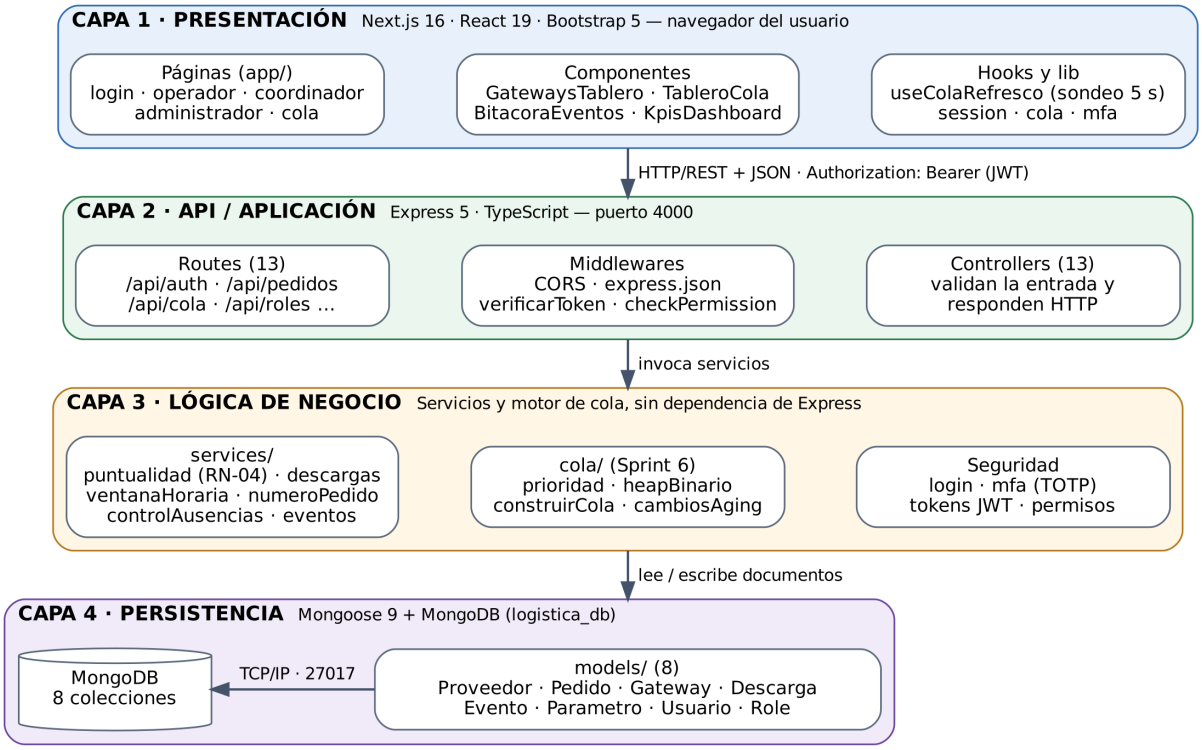


Figura 1. Vista de arquitectura en 4 capas de SumaControl.

Capa	Tecnología	Responsabilidad	Se comunica con
1. Presentación	Next.js 16, React 19, Bootstrap 5	Mostrar pantallas por rol, capturar datos y consultar la API. No contiene reglas de negocio: la decisión de qué ve cada rol la toma el backend.	Capa 2, por HTTP/REST con el JWT en la cabecera Authorization.

Capa	Tecnología	Responsabilidad	Se comunica con
2. API / aplicación	Express 5, TypeScript	Recibir la petición, autenticarla (verificarToken), autorizarla (checkPermission) y delegar. Los controladores validan la entrada y traducen el resultado a HTTP.	Capa 3.
3. Lógica de negocio	services/ y cola/	Reglas de negocio: puntualidad (RN-04), exclusividad del gateway 5 (RN-08), ventanas horarias, ciclo de descarga, MFA y el cálculo de prioridad de la cola con su heap. Son funciones que no conocen Express, por eso se prueban sin levantar el servidor.	Capa 4.
4. Persistencia	Mongoose 9, MongoDB	Definir la forma de los documentos, validarlos y leerlos o escribirlos en las 8 colecciones. El resto del sistema no habla con MongoDB directamente.	MongoDB, por TCP en el puerto 27017.

4.3 Diagrama de componentes UML

El diagrama muestra los componentes de la aplicación, las interfaces que los conectan y el orden en que atraviesa una petición el backend: primero se autentica, luego se autoriza y solo entonces llega al controlador.

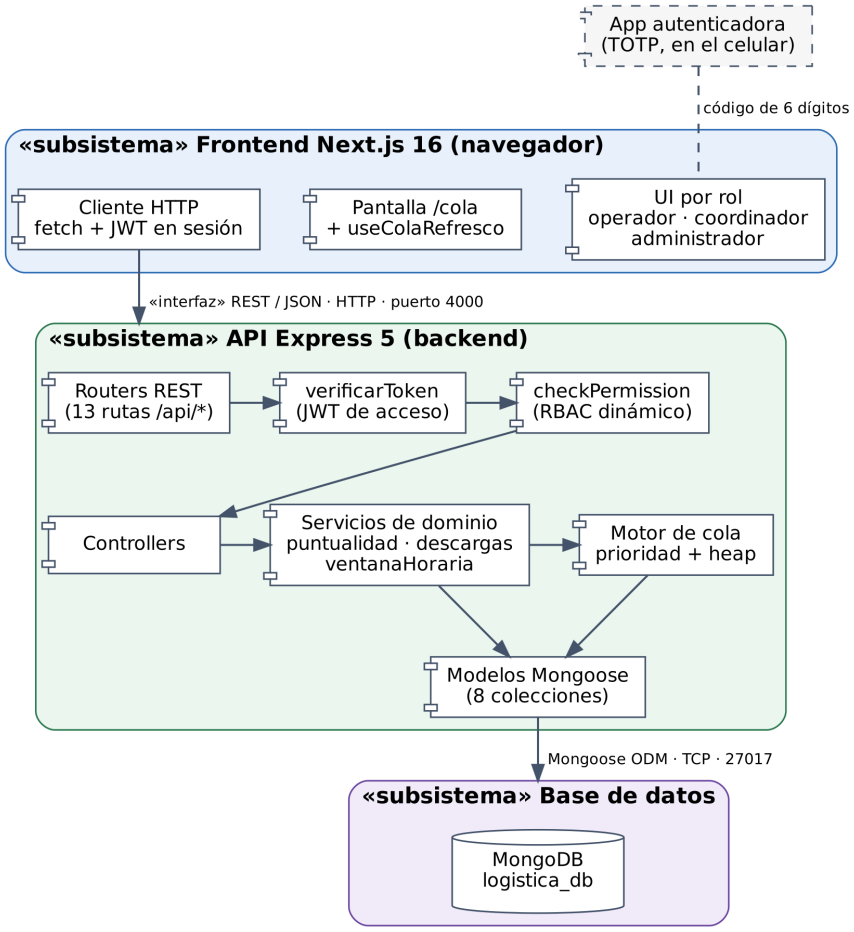


Figura 2. Diagrama de componentes UML de SumaControl.

Componente	Capa	Responsabilidad	Depende de
UI por rol	Frontend	Pantallas de operador, coordinador y administrador; el menú cambia según los permisos.	Cliente HTTP
Pantalla /cola y useColaRefresco	Frontend	Muestra la cola ordenada por prioridad y la refresca cada 5 s mientras la pestaña está visible.	GET /api/cola
Cliente HTTP	Frontend	Llama a la API con fetch y adjunta el JWT guardado en la sesión.	Routers REST
Routers REST	API	13 routers montados en /api/*: auth, mfa, usuarios, roles, proveedores, pedidos, parametros, llegadas, gateways, descargas, eventos, kpis y cola.	verificarToken
verificarToken	API	Valida la firma y el tipo del JWT de acceso; rechaza tokens temporales de MFA y tokens heredados.	checkPermission
checkPermission	API	Autorización dinámica (RBAC): recarga al usuario desde MongoDB en cada petición, une los permisos de sus roles activos y compara con los exigidos por la ruta. El JWT no decide permisos.	Controllers

Componente	Capa	Responsabilidad	Depende de
Controllers	API	Validan el cuerpo y los parámetros y responden con el código HTTP correspondiente.	Servicios de dominio
Servicios de dominio	Negocio	Puntualidad, ventana horaria, número de pedido, descargas, ausencias, eventos, MFA y tokens.	Modelos Mongoose
Motor de cola	Negocio	Calcula la prioridad (base por puntualidad más aging por espera) y ordena con un heap binario propio.	Modelos Mongoose
Modelos Mongoose	Persistencia	Esquemas, validaciones, índices y borrado lógico de las 8 colecciones.	MongoDB

4.4 Diagrama de despliegue UML

El diagrama representa los nodos de ejecución y los protocolos que el sistema usa en realidad: HTTP/REST con JSON sobre TCP/IP y actualización periódica (polling) del tablero. El sistema no usa WebSockets.

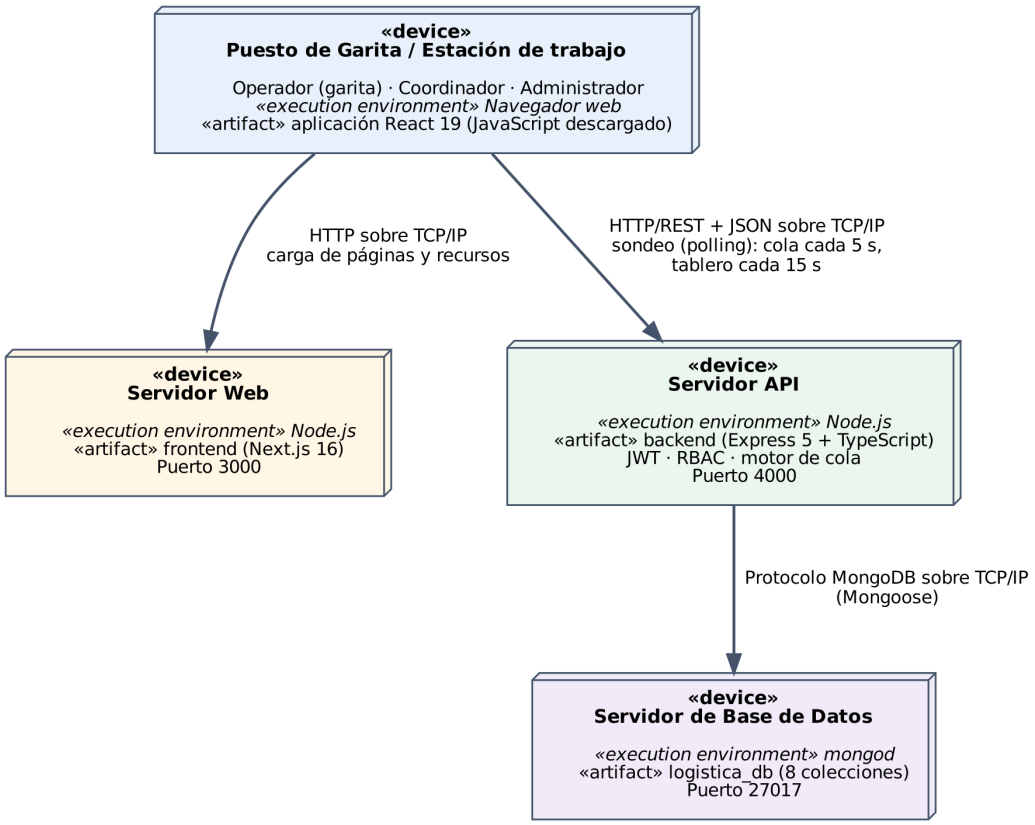


Figura 3. Diagrama de despliegue UML de SumaControl.

Nodos

Nodo	Entorno de ejecución	Artefacto y puerto	Observaciones
Puesto de garita / estación	Navegador web moderno	Aplicación React descargada del servidor web.	Operador (garita), Coordinador y Administrador usan el mismo cliente; solo cambia el rol.
Servidor web	Node.js (Next.js 16)	Frontend. Puerto 3000.	Entrega páginas y recursos estáticos; no accede a la base de datos.
Servidor API	Node.js (Express 5 + TypeScript)	Backend. Puerto 4000 (variable PORT).	Autenticación, RBAC, reglas de negocio y motor de cola. Acepta solo el origen configurado en FRONTEND_URL (CORS).
Servidor de base de datos	mongod	Base logistica_db. Puerto 27017.	Solo es alcanzable desde el servidor API, con la cadena MONGO_URI.

Protocolos y mecanismos de comunicación

Enlace	Protocolo	Uso
Navegador → Servidor web	HTTP sobre TCP/IP	Carga de páginas y recursos.

Enlace	Protocolo	Uso
Navegador → Servidor API	HTTP/REST con JSON sobre TCP/IP	Datos del negocio. Autenticación por JWT en Authorization: Bearer. Actualización periódica (polling): cola cada 5 s, tablero de gateways cada 15 s, solo con la pestaña visible.
Servidor API → MongoDB	Protocolo de MongoDB sobre TCP/IP	Acceso mediante Mongoose.
Pestañas del mismo navegador	BroadcastChannel	Aviso local de cambios entre pestañas abiertas; no sale del navegador ni reemplaza al polling.
App autenticadora → usuario	Sin conexión al servidor	El código TOTP de 6 dígitos se calcula en el celular y lo escribe el usuario.

Alcance de la topología: El repositorio no incluye infraestructura como código (contenedores, scripts de despliegue ni configuración de proxy), de modo que el diagrama muestra la topología lógica del sistema. En desarrollo y en la demostración los tres servidores corren en la misma máquina (localhost, puertos 3000, 4000 y 27017); en un despliegue real pueden separarse sin cambiar el código, porque cada nodo se localiza por variable de entorno.

Configuración de cada nodo

Nodo	Variable	Función
Servidor API	`PORT`	Puerto de escucha (por defecto 4000).
Servidor API	`MONGO_URI`	Cadena de conexión a MongoDB. Sin ella el proceso no arranca.
Servidor API	`JWT_SECRET`	Clave de firma de los tokens. Es un secreto: no va al repositorio.
Servidor API	`FRONTEND_URL`	Origen permitido por CORS (por defecto http://localhost:3000).
Servidor web	`NEXT_PUBLIC_API_URL`	Dirección base de la API que usa el navegador.

5. TRAZABILIDAD CON LOS ARTEFACTOS DEL PROYECTO

La arquitectura y los datos no son un documento aparte: son el lugar donde aterrizan los artefactos que el equipo construyó antes. La tabla indica, para cada uno, qué componentes y colecciones lo implementan. Los diagramas originales están en la carpeta docs/diagramas del repositorio.

Artefacto previo	Qué muestra	Dónde se refleja en el sistema
CU00 — Casos de uso	Actores Operador, Coordinador y Administrador y sus funciones.	Roles y permisos de la colección roles; middleware checkPermission; menú por permiso en el frontend.
DS01 — Registrar llegada	Secuencia: garita registra la llegada, el sistema clasifica la puntualidad y deja el pedido en cola.	llegadaRoutes → llegadaController → services/puntualidad (RN-04) → pedidos (estado, puntualidad, fechaHoraLlegadaReal, fechaHoraIngresoCola) y eventos (LLEGADA_REGISTRADA, INGRESO_COLA).
DS02 — Asignar gateway	Secuencia: se asigna una bahía compatible y se inicia la descarga.	descargaRoutes → services/descargas (cambio atómico LIBRE → OCUPADO con findOneAndUpdate) → gateways y descargas; eventos GATEWAY_ASIGNADO y DESCARGA_INICIADA; promoción automática desde la cola al finalizar una descarga.
Diagrama de clases	Entidades del dominio y sus relaciones.	Las 8 colecciones de la Sección 6; cada clase de dominio tiene su modelo Mongoose y las asociaciones son referencias por ObjectId.

6. PERSISTENCIA DE DATOS

6.1 Modelo de datos en MongoDB

La base de datos es MongoDB (base logistica_db), accedida con Mongoose. Al ser una base de documentos no hay tablas ni claves foráneas impuestas por el motor: la forma de los datos la imponen los esquemas Mongoose y, como segunda defensa, los validadores $jsonSchema de la Sección 6.4. La docente aceptó MongoDB y aprobó sustituir el schema.sql por estos validadores.

Decisiones de modelado

Referencias, no documentos incrustados. Las relaciones entre entidades (pedido–proveedor, descarga–gateway, evento–usuario) se guardan como ObjectId. Cada entidad cambia a su ritmo y se consulta por separado, y duplicarla dentro de otra obligaría a actualizar copias.

Arreglos incrustados solo para listas cortas y propias. usuarios.roles (referencias a roles), roles.permisos y usuarios.twoFactorRecoveryCodes pertenecen a un único documento y se leen siempre con él.

Borrado lógico. Está prohibido borrar documentos: la baja pone activo en false y las consultas filtran por activo. Así el historial y las referencias nunca quedan apuntando a la nada. La excepción es eventos, que no se da de baja jamás.

Bitácora de solo inserción. Los middlewares del modelo Evento bloquean update, replace y delete: un hecho ocurrido no se corrige, se registra otro evento.

Parámetros como texto. parametros guarda siempre el valor como string; cada módulo lo interpreta. Permite agregar parámetros de otro tipo sin migrar el esquema y ajustarlos sin redesplegar.

6.2 Diagrama entidad-relación

El diagrama muestra las 8 colecciones con sus identificadores (PK), las referencias (FK) y las claves únicas (UQ). Es un modelo lógico: MongoDB no impone las claves foráneas, las referencias son ObjectId que el código resuelve.

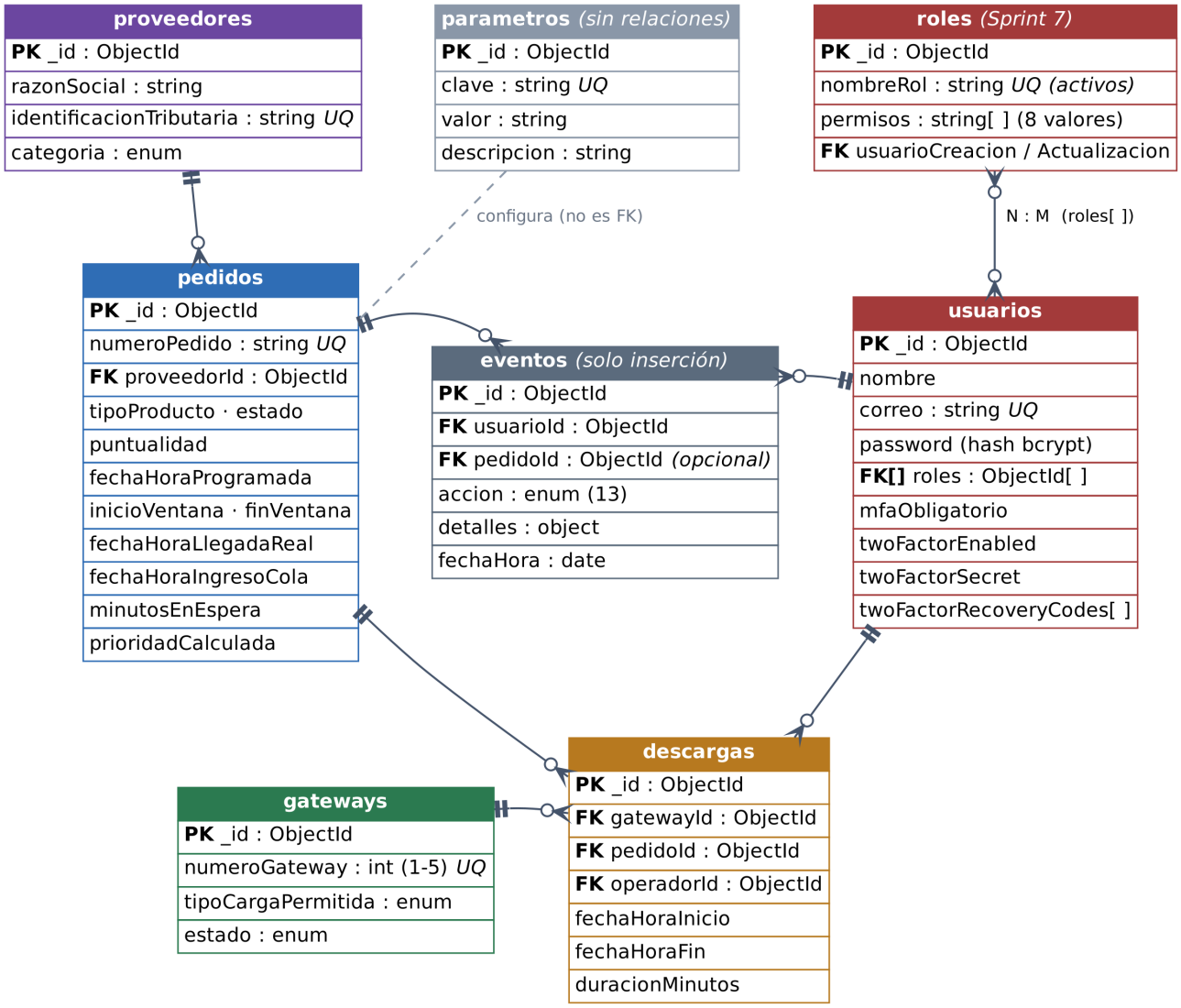


Figura 4. Diagrama entidad-relación de SumaControl (modelo lógico para MongoDB).

Relaciones entre colecciones

Relación	Cardinalidad	Campo	Tipo
proveedores → pedidos	1 : N	pedidos.proveedorId	Referencia
pedidos → descargas	1 : N	descargas.pedidoId	Referencia
gateways → descargas	1 : N	descargas.gatewayId	Referencia
usuarios → descargas	1 : N	descargas.operadorId	Referencia
usuarios → eventos	1 : N	eventos.usuarioId	Referencia
pedidos → eventos	1 : N (opcional)	eventos.pedidoId	Referencia; ausente en GATEWAY_MANTENIMIENTO
usuarios ↔ roles	N : M	usuarios.roles[]	Arreglo de referencias (Sprint 7)
usuarios → roles	1 : N	roles.usuarioCreacion / usuarioActualizacion	Referencia
parametros	—	—	Sin referencias: configuran el comportamiento del sistema

6.3 Diccionario de datos

El diccionario detalla cada campo de las 8 colecciones: tipo, restricción (PK, FK, UQ), obligatoriedad, valor por defecto y descripción funcional. "Auto" indica que lo completa el sistema y no el usuario.

Campos de auditoría comunes

Campo	Tipo	Restricción	Oblig.	Defecto	Descripción
`activo`	bool	—	No	true	Borrado lógico: false significa dado de baja.
`usuarioCreacion`	string	—	No	—	Nombre completo de quien dio el alta.
`usuarioActualizacion`	string	—	No	—	Nombre completo del último editor.
`fechaCreacion`	date	—	Auto	now	Alta del documento (la pone Mongoose).
`fechaActualizacion`	date	—	Auto	now	Última modificación (la pone Mongoose).

Campos comunes de proveedores, pedidos, gateways, descargas, parametros y usuarios (models/auditoria.ts). Las colecciones eventos y roles declaran los suyos en sus propias tablas.

Colección proveedores

Empresas que entregan mercadería en el depósito.

Campo	Tipo	Restricción	Oblig.	Defecto	Descripción
`_id`	ObjectId	PK	Auto	auto	Identificador único generado por MongoDB.
`razonSocial`	string	—	Sí	—	Nombre legal de la empresa.
`identificacionTributaria`	string	UQ	Sí	—	Identificación tributaria; no puede repetirse.
`categoria`	string (enum)	construcción | general	Sí	—	Tipo de carga que entrega; se normaliza a NFC.
`contactoNombre`	string	—	Sí	—	Persona de contacto.
`telefono`	string	—	Sí	—	Teléfono de contacto.
`emailContacto`	string	formato email	Sí	—	Correo de contacto, en minúsculas.

Colección pedidos

Entregas programadas y su recorrido: llegada, cola y descarga.

Campo	Tipo	Restricción	Oblig.	Defecto	Descripción
`_id`	ObjectId	PK	Auto	auto	Identificador único generado por MongoDB.
`numeroPedido`	string	UQ	Sí	generado	Código PED-EQUI04-000000006; lo genera el servidor.
`proveedorId`	ObjectId	FK → proveedores	Sí	—	Proveedor que realiza la entrega.
`tipoProducto`	string (enum)	construcción | general	Sí	—	Tipo de carga; decide el gateway compatible.
`fechaHoraProgramada`	date	—	Sí	—	Hora de la cita.
`duracionEstimadaMinutos`	number	mín. 1	Sí	—	Duración estimada de la descarga.
`inicioVentana`	date	—	Sí	calculado	Inicio de la ventana; lo calcula el servidor.
`finVentana`	date	> inicioVentana	Sí	calculado	Fin de la ventana; debe ser posterior al inicio.

Campo	Tipo	Restricción	Oblig.	Defecto	Descripción
estado	string (enum)	9 valores	No	PROGRAMADO	PROGRAMADO, ANTICIPADO, A TIEMPO, TARDÍO, AUSENTE, CANCELADO, EN COLA, DESCARGANDO, FINALIZADO.
puntualidad	string (enum)	4 valores	No	—	ANTICIPADO, A TIEMPO, TARDÍO o AUSENTE. Vacía hasta registrar la llegada.
fechaHoraLlegadaReal	date	—	No	—	Llegada real; la pone el servidor.
fechaHoraIngresoCola	date	—	No	—	Entrada a la cola (Sprint 6).
minutosEnEspera	number	—	No	0	Minutos en cola en la última reevaluación (Sprint 6).
prioridadCalculada	number	—	No	—	Prioridad con aging aplicado; mayor número, mayor prioridad (Sprint 6).

Colección gateways

Las cinco bahías físicas de descarga.

Campo	Tipo	Restricción	Oblig.	Defecto	Descripción
_id	ObjectId	PK	Auto	auto	Identificador único generado por MongoDB.
numeroGateway	number (int)	UQ, 1 a 5	Sí	—	Número de la bahía; no se edita.
tipoCargaPermitida	string (enum)	construcción | general	Sí	—	Bahías 1 a 4: general. Bahía 5: construcción (RN-07, RN-08).
estado	string (enum)	LIBRE | OCUPADO | FUERA DE SERVICIO	No	LIBRE	Disponibilidad actual.

Colección descargas

Ocupación de un gateway por un pedido.

Campo	Tipo	Restricción	Oblig.	Defecto	Descripción
_id	ObjectId	PK	Auto	auto	Identificador único generado por MongoDB.
gatewayId	ObjectId	FK → gateways	Sí	—	Bahía utilizada.
pedidoId	ObjectId	FK → pedidos	Sí	—	Pedido descargado.
operadorId	ObjectId	FK → usuarios	Sí	—	Quien registró el inicio.
fechaHoraInicio	date	—	Sí	—	Inicio; la pone el servidor.
fechaHoraFin	date | null	—	No	null	Fin; null mientras la descarga está en curso.
duracionMinutos	number | null	—	No	null	Minutos redondeados entre inicio y fin; los calcula el servidor.

Colección eventos

Bitácora de auditoría de solo inserción (RN-14).

Campo	Tipo	Restricción	Oblig.	Defecto	Descripción
_id	ObjectId	PK	Auto	auto	Identificador único generado por MongoDB.
usuarioId	ObjectId	FK → usuarios	Sí	—	Quien provocó el evento; sale del token.
pedidoId	ObjectId	FK → pedidos	No	—	Pedido afectado, si aplica.

Campo	Tipo	Restricción	Oblig.	Defecto	Descripción
accion	string (enum)	13 valores	Sí	—	LLEGADA_REGISTRADA, GATEWAY_ASIGNADO, DESCARGA_INICIADA, DESCARGA_FINALIZADA, CITA_REPROGRAMADA, GATEWAY_MANTENIMIENTO, INGRESO_COLA, AGING_APLICADO, PROMOCION_AUTOMATICA_GATEWAY, ROL_CREADO, ROL_MODIFICADO, ROL_DESACTIVADO, ROL_ASIGNADO_USUARIO.
detalles	object	—	No	{ }	Datos propios de cada acción; forma libre a propósito.
fechaHora	date	—	Sí	now	Momento del hecho; la pone el servidor.
activo	bool	—	No	true	Nunca cambia: un evento no se da de baja.
usuarioCreacion	string	—	No	—	Nombre completo de quien provocó el evento.
fechaCreacion	date	—	Auto	now	Alta del documento. No tiene fechaActualizacion.

Colección parametros

Configuración del sistema editable sin redesplegar.

Campo	Tipo	Restricción	Oblig.	Defecto	Descripción
_id	ObjectId	PK	Auto	auto	Identificador único generado por MongoDB.
clave	string	UQ	Sí	—	Nombre del parámetro, en mayúsculas.
valor	string	—	Sí	—	Valor como texto; cada módulo lo interpreta.
descripcion	string	—	No	—	Qué controla el parámetro.

Colección usuarios

Cuentas de acceso, con segundo factor opcional.

Campo	Tipo	Restricción	Oblig.	Defecto	Descripción
_id	ObjectId	PK	Auto	auto	Identificador único generado por MongoDB.
nombre	string	—	Sí	—	Nombre completo.
correo	string	UQ, formato email	Sí	—	Usuario de acceso, en minúsculas.
password	string	—	Sí	—	Hash bcrypt. No se devuelve en consultas ni en JSON.
roles	ObjectId[ ]	FK → roles; mín. 1, sin repetir	Sí	—	Roles asignados (Sprint 7); reemplaza al antiguo campo rol.
mfaObligatorio	bool	—	No	false	El administrador exige segundo factor a este usuario.
twoFactorSecret	string	—	No	—	Secreto TOTP en Base32. Secreto: no se devuelve.
twoFactorEnabled	bool	—	No	false	El usuario ya confirmó su segundo factor.
twoFactorRecoveryCodes	string[ ]	—	No	[ ]	Hashes bcrypt de los códigos de recuperación de un solo uso.

Colección roles

Roles dinámicos con sus permisos (Sprint 7).

Campo	Tipo	Restricción	Oblig.	Defecto	Descripción
`_id`	ObjectId	PK	Auto	auto	Identificador único generado por MongoDB.
`nombreRol`	string	UQ entre activos; 3 a 60	Sí	—	Nombre del rol; único sin distinguir mayúsculas.
`descripcion`	string	máx. 200	No	—	Para qué sirve el rol.
`permisos`	string[ ]	enum de 8; mín. 1, sin repetir	Sí	—	usuarios, roles, proveedores, pedidos, gateways, descargas, cola, kpis.
`activo`	bool	—	No	true	Baja lógica del rol.
`usuarioCreacion`	ObjectId	FK → usuarios	No	—	Quien creó el rol.
`usuarioActualizacion`	ObjectId	FK → usuarios	No	—	Quien lo modificó por última vez.
`fechaCreacion`	date	—	Auto	now	Alta del documento.
`fechaActualizacion`	date	—	Auto	now	Última modificación.

6.4 Validadores $jsonSchema

Cada colección tiene un validador $jsonSchema derivado de su esquema Mongoose. El archivo schema.js los aplica y crea los índices de la tabla anterior; es idempotente (usa collMod si la colección ya existe y createCollection si no) y se ejecuta con mongosh. Los validadores trabajan con validationLevel "moderate", para no rechazar documentos antiguos que se actualicen sin tocar los campos nuevos, y validationAction "error", que rechaza las escrituras inválidas.

El script se ejecuta así, desde la carpeta que lo contiene:

```
mongosh "mongodb://localhost:27017/logistica_db" --file schema.js
```

Ejemplo: validador de la colección pedidos (fragmento de schema.js)

```
pedidos: {
  bsonType: 'object',
  required: [
    'numeroPedido', 'proveedorId', 'tipoProducto', 'fechaHoraProgramada',
    'duracionEstimadaMinutos', 'inicioVentana', 'finVentana'
  ],
  properties: {
    numeroPedido: { bsonType: 'string', description: 'Ej. PED-EQUI04-000000006; unico' },
    proveedorId: { bsonType: 'objectId', description: 'FK -> proveedores._id' },
    tipoProducto: { enum: CATEGORIAS },
    fechaHoraProgramada: { bsonType: 'date' },
    duracionEstimadaMinutos: { bsonType: ['int', 'long', 'double'], minimum: 1 },
    inicioVentana: { bsonType: 'date' },
    finVentana: { bsonType: 'date' },
    estado: { enum: ESTADOS_PEDIDO, description: 'Por defecto PROGRAMADO' },
    puntualidad: { enum: PUNTUALIDADES },
    fechaHoraLlegadaReal: { bsonType: 'date' },
    fechaHoraIngresoCola: { bsonType: 'date', description: 'Sprint 6' },
    minutosEnEspera: { bsonType: ['int', 'long', 'double'], description: 'Sprint 6; por defecto 0' },
    prioridadCalculada: { bsonType: ['int', 'long', 'double'], description: 'Sprint 6' },
    ...auditoria()
  }
},
```

Reglas que el validador no puede expresar

$jsonSchema valida la forma de un documento, no relaciones entre campos ni el comportamiento de las operaciones. Estas reglas siguen en Mongoose y en los controladores:

Colección	Regla	Dónde se aplica
pedidos	finVentana debe ser posterior a inicioVentana	Hook pre-validate del modelo Pedido y validación del controlador.
gateways	El tipo de carga debe coincidir con el número de bahía (RN-07, RN-08)	Hook pre-validate del modelo Gateway y validación del controlador.
eventos	Solo inserción: no se modifica ni se elimina	Middlewares del modelo Evento; no existen rutas de escritura sobre /api/eventos.
usuarios	password se almacena como hash bcrypt	Hook pre-save del modelo Usuario; el validador solo comprueba que sea texto.

6.5 Índices

Los índices responden a las consultas reales del sistema; los compuestos siguen el orden de la consulta (campos de igualdad primero, el de rango u orden al final).

Colección	Índice	Tipo	Para qué
proveedores	`{ identificacionTributaria: 1 }`	Único	Evita proveedores duplicados.
pedidos	`{ numeroPedido: 1 }`	Único	Búsqueda y unicidad del código de pedido.
pedidos	`{ activo, estado, inicioVentana }`	—	Control de solapamiento de ventanas al programar.
pedidos	`{ activo, estado, tipoProducto, fechaHoraLlegadaReal }`	—	Promoción de la cola al liberarse un gateway.
gateways	`{ numeroGateway: 1 }`	Único	Una bahía por número.
gateways	`{ activo, numeroGateway }`	—	Listado ordenado de bahías activas.
descargas	`{ activo, fechaHoraFin }`	—	Descargas activas (fechaHoraFin nula).
eventos	`{ pedidoId, fechaHora: -1 }`	—	Historial de un pedido.
eventos	`{ accion, fechaHora: -1 }`	—	Bitácora filtrada por acción.
eventos	`{ fechaHora: -1 }`	—	Bitácora reciente.
parametros	`{ clave: 1 }`	Único	Un valor por parámetro.
usuarios	`{ correo: 1 }`	Único	Inicio de sesión y unicidad del correo.
roles	`{ nombreRol: 1 }`	Único parcial (activo: true), collation es nivel 2	Nombre único entre roles activos y reutilizable tras una baja.

6.6 Parámetros de configuración

La colección parametros concentra los valores que el negocio puede ajustar sin redesplegar. Son también la primera palanca del mantenimiento adaptativo.

Clave	Valor inicial	Función
`HORA_APERTURA`	7	Hora de apertura del depósito.
`HORA_CIERRE`	17	Hora de cierre del depósito.
`DIAS_HABILES`	1,2,3,4,5,6	Días operativos (1 = lunes … 6 = sábado).
`TOLERANCIA_ANTICIPADO_MINUTOS`	15	Minutos antes del inicio de la ventana a partir de los cuales (más de) la llegada es ANTICIPADO.
`TOLERANCIA_TARDIO_MINUTOS`	15	Minutos después del inicio de la ventana que todavía se consideran A TIEMPO.
`TOLERANCIA_AUSENTE_MINUTOS`	60	Minutos después del inicio de la ventana a partir de los cuales el pedido pasa a AUSENTE.
`NUMERO_EQUIPO`	04	Número de equipo que forma parte del código de pedido.
`FACTOR_AGING_MIN`	20	Minutos de espera que suman un punto de prioridad (Sprint 6).

Clave	Valor inicial	Función
LIMITE_ESPERA_MAX	60	Minutos de espera a partir de los cuales la cola marca el pedido como excedido (Sprint 6).

6.7 Datos derivados del motor de cola

El motor de cola no tiene tablas propias: persiste su resultado en tres campos de pedidos (fechaHoraIngresoCola, minutosEnEspera y prioridadCalculada) y deja huella en eventos. La prioridad se calcula como la base de la puntualidad (A TIEMPO = 3, ANTICIPADO = 2, TARDÍO = 1) más un punto por cada bloque de FACTOR_AGING_MIN minutos de espera, de modo que ningún pedido queda postergado indefinidamente. La cola se ordena por prioridad descendente y, ante empate, por hora programada, por hora de ingreso a la cola y por _id, lo que hace el orden determinista.

Para L3 conviene recordar la convención: en SumaControl un número mayor significa mayor prioridad, a diferencia del servicio de referencia del curso de Estructuras de Datos, donde 1 era la prioridad más alta.

7. OBSERVACIONES Y LIMITACIONES

Al consolidar la documentación contra el código aparecieron diferencias y pendientes. Se consignan aquí para que quien reciba el sistema sepa exactamente qué está hecho y qué no.

OBS-01 — El Sprint 7 vive todavía en su rama

Los roles dinámicos (colección roles, checkPermission, usuarios.roles) están en la rama sprint7-victor y no en develop. Este informe los documenta tal como están en esa rama. Si el modelo cambia antes de integrarse, el diccionario, el DER y schema.js deben regenerarse a partir de models/Role.ts y models/Usuario.ts.

OBS-02 — La consigna menciona tecnologías que el proyecto no usa

La consigna cita PostgreSQL, vacuum, WebSockets y schema.sql. El proyecto usa MongoDB, HTTP/REST con polling y validadores $jsonSchema. La docente aprobó esa adaptación en su aclaración de entregables, y el informe describe lo implementado.

OBS-03 — Faltan piezas de infraestructura que el plan da por definidas

Respaldos automáticos, supervisor de procesos, monitoreo y medición de latencia están definidos en la Sección 3 pero no existen en el repositorio (ver el estado actual en la tabla de la Sección 3.5). El plan los declara como acciones por implementar y no como capacidades vigentes.

OBS-04 — Hay reglas que un $jsonSchema no puede expresar

Cuatro reglas siguen dependiendo de Mongoose y de los controladores (tabla de la Sección 6.4). El validador es una segunda defensa, no un reemplazo.

OBS-05 — Los tokens anteriores al Sprint 7 se rechazan

verificarToken rechaza los tokens que no llevan el claim de permisos, de modo que una sesión abierta antes de integrar el Sprint 7 recibe un 401 y debe iniciar sesión otra vez. Es un caso típico de ticket L1: la respuesta es volver a ingresar, no escalar.

OBS-06 — Migración de usuarios al modelo de roles dinámicos

El campo rol (texto) fue reemplazado por roles (arreglo de referencias). El script backend/src/scripts/migrarRoles.ts convierte los usuarios existentes; debe ejecutarse una sola vez al desplegar el Sprint 7 y forma parte del procedimiento de despliegue de L2.

8. CIERRE Y DECLARACIÓN

Con este informe SumaControl queda con un esquema de soporte de tres niveles, con responsables, tiempos y contingencia para cada severidad; con una estrategia de mantenimiento para sus cuatro dimensiones, basada en los comandos y la configuración reales del proyecto; y con su arquitectura y su modelo de datos consolidados y trazables hasta los artefactos que el equipo construyó antes. Lo que todavía no existe se declara como pendiente en lugar de darse por hecho.

8.1 Declaración de uso de Inteligencia Artificial (estándar FEEN)

📌 **DECLARACIÓN OBLIGATORIA:** para la elaboración de este informe el equipo utilizó las herramientas de Inteligencia Artificial Claude y ChatGPT con tres propósitos acotados: (1) recorrer el código del repositorio para verificar que cada afirmación del documento corresponde al comportamiento real del sistema; (2) apoyar la redacción y la ortografía; y (3) organizar en tablas y diagramas la información producida por el equipo. El esquema de soporte, los tiempos de respuesta, las estrategias de mantenimiento y las decisiones de arquitectura fueron definidos y validados por los integrantes del Equipo Scrum. El diccionario de datos y los validadores se derivaron de los esquemas Mongoose del repositorio y se revisaron contra ellos. El equipo asume la responsabilidad técnica completa de lo aquí documentado, de conformidad con los Lineamientos del Uso de la Inteligencia Artificial de la FEEN / UJMD.